

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 04: Range Queries

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
6 de noviembre de 2026

Índice

1. Resumen **1**
1.1. Propósito y alcance del estudio . . . 1
1.2. Estructuras y algoritmos estudiados . . . 1

2. Análisis teórico **2**
2.1. Sparse Table . . . 2
2.2. Segment Tree . . . 3
2.3. Comparación de complejidades . . . 3

3. Implementación **4**
3.1. Sparse Table . . . 4
3.2. Segment Tree . . . 4

4. Metodología experimental **5**
4.1. Características de la máquina . . . 5
4.2. Datos de entrada . . . 5
4.3. Experimentos e hipótesis . . . 6
4.3.1. Construcción inicial en el escenario estático . . . 7
4.3.2. Consultas en el escenario estático . . . 7
4.3.3. Mantenimiento en el escenario dinámico . . . 8
4.3.4. Consultas en el escenario dinámico . . . 8
4.3.5. Análisis del espacio utilizado . . . 8

Referencias **10**

1. Resumen

1.1. Propósito y alcance del estudio

Este estudio compara dos estructuras de datos para resolver el problema de *Range Minimum Query* (RMQ): **Sparse Table** y **Segment Tree**. Dado un arreglo de enteros A, una consulta sobre el rango inclusivo $A[L..R]$ busca obtener el **índice del elemento mínimo**, no solamente su valor. Se consideran un **escenario estático**, en el que el arreglo permanece fijo, y un **escenario dinámico**, en el que sus elementos reciben modificaciones puntuales.

El diseño experimental contempla cuatro comparaciones:

- **Construcción en el escenario estático:** medir el tiempo necesario para construir ambas estructuras a partir del mismo arreglo, considerando diferentes cantidades de elementos.
- **Consultas en el escenario estático:** medir el tiempo de respuesta sobre las estructuras ya construidas, utilizando rangos aleatorios de distintas longitudes y arreglos de diferentes tamaños.
- **Mantenimiento en el escenario dinámico:** comparar el tiempo de una actualización puntual en Segment Tree con el tiempo de reconstruir Sparse Table después del mismo cambio. Esta comparación representa el costo de mantener respuestas vigentes, no dos operaciones de actualización equivalentes.
- **Consultas en el escenario dinámico:** medir el tiempo de respuesta después de aplicar los cambios y mantener ambas estructuras actualizadas, separándolo del costo de mantenimiento.

En cada comparación se utilizarán las mismas entradas y consultas para ambas estructuras. Además, se realizará un **análisis del espacio utilizado**, considerando las asignaciones de memoria de los constructores y las reservas temporales relevantes durante la construcción o reconstrucción. Los resultados se interpretarán junto con las complejidades teóricas para discutir las ventajas y limitaciones de cada alternativa. El estudio también abordará su adaptación a otras operaciones binarias y las propiedades algebraicas que permiten conservar sus garantías.

1.2. Estructuras y algoritmos estudiados

- **Sparse Table:** preprocesa el arreglo almacenando resultados para bloques de longitud potencia de dos. Para RMQ permite responder consultas en tiempo constante mediante la combinación de dos bloques que pueden solaparse. La implementación estudiada no ofrece actualizaciones, por lo que en el escenario dinámico se reconstruirá después de cada modificación.
- **Segment Tree:** organiza los mínimos de intervalos en una estructura jerárquica. Responde consultas y permite actualizar un elemento en tiempo logarítmico, recalculando los nodos afectados sin reconstruir toda la estructura.

Las implementaciones incorporadas proceden del repositorio oficial del curso, con autoría original de LELE [1, 2].

2. Análisis teórico

Sea A un arreglo no vacío de n enteros. Una consulta RMQ sobre $A[L..R]$, con $0 \le L \le R < n$, devuelve el índice de su elemento mínimo. Denotamos por $k = R - L + 1$ la longitud del rango y resolvemos los empates con el **menor índice**. Para ello, cada candidato se representa mediante el par $(A[i], i)$: el mínimo lexicográfico compara primero el valor y luego el índice. Las implementaciones reciben intervalos semiabiertos $[L, R+1)$; convertir los extremos y extraer el índice de la respuesta cuesta $\Theta(1)$.

Usamos $O(\cdot)$ para cotas superiores y $\Theta(\cdot)$ para cotas ajustadas. Suponemos un modelo de palabras de máquina: valores, índices y pares ocupan espacio constante, y acceder, copiar y comparar un par cuesta $\Theta(1)$. El espacio de la estructura incluye sus copias internas, pero excluye el arreglo externo de entrada; devolver un índice ocupa $\Theta(1)$. Las expresiones con $\log n$, en base dos, se presentan para $n \ge 2$; con un solo elemento las operaciones RMQ son constantes. El análisis se deriva del código del curso y de las descripciones de ambas estructuras [1, 2, 3, 4].

2.1. Sparse Table

Sparse Table almacena respuestas para bloques de longitud potencia de dos. Si $S_j[i]$ es el mínimo lexicográfico del bloque que comienza en i y tiene 2^j elementos, su construcción utiliza

$$S_0[i] = (A[i], i), \qquad S_j[i] = \min_{\text{lex}}\big(S_{j-1}[i], S_{j-1}[i + 2^{j-1}]\big).$$

La implementación reserva $\ell = \lfloor \log_2 n \rfloor + 1$ filas completas de n elementos, aunque algunas posiciones no correspondan a bloques válidos [1]. Para una consulta, toma $b = \lfloor \log_2 k \rfloor$ y combina

$$\min_{\text{lex}}\big(S_b[L], S_b[R - 2^b + 1]\big).$$

Los dos bloques cubren el rango y pueden solaparse. La duplicación de su intersección no cambia el mínimo, por la idempotencia de esta operación [3].

- **Preparación (construcción):** $\Theta(n \log n)$, tanto en mejor como en peor caso. Inicializar las filas ya requiere $\Theta(n\ell)$ trabajo. Además, si $m = \lfloor \log_2 n \rfloor$, se realizan $\sum_{j=1}^{m}(n - 2^j + 1) = m(n+1) - 2^{m+1} + 2$ combinaciones; cada una cuesta tiempo constante.
- **Mejor caso de consulta RMQ:** $\Theta(1)$.
- **Peor caso de consulta RMQ:** $\Theta(1)$. El método `query_idempotent` realiza dos accesos y una combinación. Calcula el nivel con `std::bit_width`, que permite obtener $\lfloor \log_2 k \rfloor$ sobre una palabra de ancho fijo sin recorrer el rango [3, 6].
- **Consulta general:** el método distinto `query` descompone k en potencias de dos y combina bloques disjuntos. Procesa tantos bloques como bits uno tiene k: su costo es $\Theta(\text{popcount}(k))$, con mejor caso $\Theta(1)$ y cota $O(\log(k+1))$. No es la variante utilizada para medir RMQ.
- **Mantenimiento tras un cambio:** la implementación no ofrece `update`. En el escenario dinámico se reconstruye la tabla, con costo $\Theta(n \log n)$ por modificación, independientemente de su posición. Es una estrategia de mantenimiento del estudio, no una operación de actualización nativa ni una imposibilidad de diseñar otras variantes.
- **Memoria:** $\Theta(n \log n)$ persistente y $\Theta(1)$ auxiliar por consulta. Para $s = \text{sizeof}(T)$, las filas contienen $n\ell$ objetos de tipo T, cuyo tamaño lógico de datos es $n\ell s$ bytes, además de los metadatos de los vectores y del combinador. La expresión que crea las filas utiliza también un vector temporal de n elementos: $\Theta(n)$ espacio temporal, sin cambiar el orden $\Theta(n \log n)$ del pico total de construcción.

El tamaño lógico no debe confundirse con los bytes efectivamente reservados: estos dependen de las capacidades de los vectores y de la biblioteca usada. Si durante una reconstrucción coexisten la tabla anterior y la nueva, el pico puede aumentar, pero conserva el mismo orden asintótico.

2.2. Segment Tree

Segment Tree mantiene mínimos de grupos de elementos en una jerarquía binaria [4]. La versión del curso es iterativa: reserva un vector de $2n$ posiciones, coloca las hojas en $[n, 2n)$ y calcula los nodos internos combinando sus dos hijos. No redondea n a una potencia de dos [2].

- **Preparación (construcción):** $\Theta(n)$, tanto en mejor como en peor caso. Inicializa $2n$ posiciones, copia n hojas y calcula exactamente $n-1$ mínimos de hijos. No realiza n actualizaciones independientes, lo que costaría $O(n \log n)$.
- **Mejor caso de consulta:** $\Theta(1)$, por ejemplo al consultar un único elemento. En ese caso el intervalo se consume en una iteración del recorrido ascendente.
- **Peor caso de consulta:** $\Theta(\log n)$. Los dos extremos ascienden dividiéndose por dos y se incorporan como máximo dos nodos por nivel. Existen rangos que requieren recorrer $\Theta(\log n)$ niveles. Más precisamente, para longitud k el bucle queda acotado por $O(\log(k+1))$, porque la cantidad de posiciones pendientes se reduce al menos a la mitad en cada paso. Por ello, mantener k fijo no obliga a observar crecimiento logarítmico con n, aunque sí puedan variar los accesos a caché.
- **Actualización puntual:** $\Theta(\log n)$ en mejor y peor caso para $n \geq 2$. Después de escribir una hoja, el método `update` recalcula todos sus ancestros hasta la raíz. No termina anticipadamente si el nuevo valor coincide con el anterior o si un mínimo intermedio no cambia.
- **Memoria:** $\Theta(n)$ persistente y $\Theta(1)$ auxiliar por consulta y actualización. Los $2n$ objetos del vector representan $2ns$ bytes lógicos de datos, más los metadatos y cualquier capacidad adicional. La construcción usa $\Theta(1)$ espacio temporal fuera de ese almacenamiento, y el pico total es $\Theta(n)$. Las actualizaciones no crean nuevos nodos.

Estas cotas se refieren al código iterativo incorporado, no a otra implementación recursiva que reserve $4n$ posiciones. En ambas estructuras, el contenido de los valores no modifica las reservas principales ni la cantidad de pasos de construcción; el costo de consulta del árbol depende del rango, no de encontrar antes un valor especialmente pequeño.

2.3. Comparación de complejidades

La tabla resume RMQ con pares de tamaño fijo. La consulta excluye la construcción y el mantenimiento; en Sparse Table corresponde a `query_idempotent`. La columna de mantenimiento compara reconstruir Sparse Table con actualizar Segment Tree, no dos constructores.

Estructura	Construcción	Mejor consulta	Peor consulta	Mantenimiento	Memoria persistente
Sparse Table	$\Theta(n \log n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Segment Tree	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$

3. Implementación

Las dos implementaciones son clases genéricas de C++20 obtenidas del repositorio del curso y conservadas en `include/range_queries/`. Reciben un vector de elementos de tipo `T` y consultan intervalos semiabiertos $[i, j)$. Para obtener el índice del mínimo se utilizarán pares (valor, índice), cuyo orden lexicográfico resuelve empates por el menor índice. Se reemplazaron los comentarios y se añadió la cabecera `<bit>` a Sparse Table, sin modificar la lógica de los algoritmos.

3.1. Sparse Table

La clase `sparse_table` guarda una matriz de vectores `ST` y una función de combinación `f`. El constructor obtiene $m = \lfloor \log_2 n \rfloor$ mediante `_lg`, reserva $m+1$ filas de n elementos y copia la entrada en `ST[0]`. Dos ciclos construyen las filas siguientes: `p` representa la longitud de un bloque y `q` el doble; cada posición válida combina dos bloques consecutivos de la fila anterior.

Para RMQ se empleará `query_idempotent`, que calcula el nivel `b` a partir de la longitud del intervalo y combina los bloques de esa fila que comienzan en i y $j - 2^b$. La clase también incluye `query`, que recorre bloques disjuntos, pero no es la variante seleccionada para esta comparación. Como no dispone de `update`, en el escenario dinámico se reconstruirá desde el arreglo modificado. La implementación proviene del repositorio del curso [1].

3.2. Segment Tree

La clase `segment_tree` almacena la estructura en un vector `ST` de $2n$ elementos. El constructor copia la entrada en las hojas, desde `ST[n]` hasta `ST[2n - 1]`, y recorre los índices internos en orden descendente para guardar el mínimo de cada par de hijos mediante `_min`.

El método `query` desplaza los extremos del intervalo a las hojas e inicializa `q` con un elemento del rango. Un ciclo asciende por el árbol, incorpora los nodos de los bordes según la paridad de los extremos y conserva el menor candidato. El método `update` cambia una hoja y recalcula el mínimo de sus ancestros, combinando cada nodo con su hermano hasta alcanzar la raíz, sin reconstruir el vector. La implementación proviene del repositorio del curso [2].

4. Metodología experimental

4.1. Características de la máquina

Los experimentos se realizarán en una estación de trabajo con las siguientes características:

- **Procesador:** Intel Core i5-13420H (Raptor Lake), 8 núcleos y 12 hilos (4 P-Cores y 4 E-Cores), frecuencia base de 2.10 GHz y turbo de hasta 4.60 GHz. Cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB.
- **Memoria principal:** 16 GiB DDR (15 GiB utilizables por el sistema operativo).
- **Almacenamiento:** SSD NVMe WD PC SN740 de 512 GB, interfaz PCIe Gen4 $\times 4$.
- **Entorno de compilación:** Arch Linux x86_64 (kernel 6.x), GCC y las banderas `-std=c++20 -O3 -march=native`.
- **Afinidad de CPU:** se fijará el proceso de medición al núcleo lógico 2 mediante `taskset -c 2`. Esto reduce las migraciones entre núcleos, pero no elimina variaciones de frecuencia, temperatura ni la actividad concurrente del sistema.

4.2. Datos de entrada

Se utilizarán **arreglos de enteros generados artificialmente**. Para cada tamaño n se generarán tres arreglos con valores de distribución uniforme en el intervalo $[0, 10^9]$, utilizando las semillas fijas **1729, 2718 y 31415**. Se registrarán las semillas y los parámetros de generación para reproducir las entradas. Sparse Table y Segment Tree recibirán exactamente los mismos datos en cada comparación.

- **Representación de los elementos:** cada entero se asociará con su posición mediante un par $(\text{valor}, \text{índice})$. Ambos componentes serán enteros: el dato numérico y su posición. El orden lexicográfico permitirá obtener el índice del mínimo y, ante valores iguales, seleccionar la posición más a la izquierda. La preparación de estos pares quedará fuera del tiempo medido.
- **Consultas:** para una longitud de rango k, con $1 \leq k \leq n$, se elegirá uniformemente una posición inicial L entre 0 y $n-k$, y se fijará $R = L+k-1$. Así, cada consulta corresponderá a un intervalo inclusivo válido $[L, R]$, que se traducirá a $[L, R+1)$ al invocar las implementaciones. Se considerarán distintas longitudes y ambas estructuras responderán la misma secuencia de consultas.
- **Actualizaciones:** en el escenario dinámico se generará una secuencia reproducible de posiciones elegidas uniformemente entre 0 y $n-1$, permitiendo posiciones repetidas, y nuevos valores del intervalo $[0, 10^9]$, distintos del valor vigente. Los cambios se aplicarán en el mismo orden: Segment Tree actualizará la hoja correspondiente y Sparse Table se reconstruirá a partir del arreglo modificado. El índice asociado a cada elemento se conservará. Las consultas posteriores serán iguales para ambas alternativas.
- **Entradas de validación:** se emplearán, por separado, arreglos pequeños crecientes, decrecientes, con todos sus elementos iguales, con mínimos repetidos y de un solo elemento. Estos casos comprobarán la corrección y el desempate; no constituirán experimentos temporales adicionales.

La generación de los arreglos, las consultas y la secuencia de modificaciones se realizará antes del cronometraje. Los tamaños de arreglo, las longitudes concretas de los rangos y las cantidades de operaciones se fijarán antes de ejecutar los experimentos.

4.3. Experimentos e hipótesis

Se compararán Sparse Table y Segment Tree mediante **cuatro experimentos**: construcción inicial y consultas en el escenario estático, y mantenimiento y consultas posteriores a modificaciones en el escenario dinámico. Para cada entrada se construirán ambas estructuras y se **reutilizarán las instancias** para consultar el arreglo sin cambios. Después se aplicará la secuencia de modificaciones: Segment Tree conservará su instancia mediante `update`, mientras que Sparse Table será sustituida por una nueva construcción del arreglo vigente. No se reconstruirán estructuras entre consultas si no ha ocurrido un cambio.

El cronometraje seguirá el esquema de la plantilla `uhr`, de Leonardo Lovera (LELE), adaptado para usar `std::chrono::steady_clock` y expresar los tiempos en nanosegundos [9]. Se realizarán **32 repeticiones por configuración y semilla**, después del calentamiento. Las consultas estáticas se medirán en lotes calibrados fuera de muestra, conservando el tamaño del lote entre repeticiones de una misma configuración y método. En el escenario dinámico se respetará la carga fija definida a continuación, sin añadir consultas para calibrar el reloj. El tiempo de cada lote se dividirá por la cantidad de consultas ejecutadas. La construcción y el mantenimiento se expresarán por construcción y por modificación, respectivamente, sin mezclarlos con el tiempo de consulta.

La generación de datos, preparación de pares, validación y escritura de resultados quedarán fuera del reloj. También se excluirá la destrucción de las estructuras, tanto después de construir como al reemplazar Sparse Table; en este último caso se cronometrará la creación de la tabla nueva. Los índices devueltos se consumirán para evitar que el compilador elimine las consultas. La corrección se contrastará con un barrido lineal independiente, fuera de los intervalos medidos.

En cada repetición dinámica se partirá del mismo arreglo inicial y se reproducirán los mismos cambios y consultas. La restauración del estado y el calentamiento quedarán fuera de la muestra; dentro de la secuencia se conservarán las instancias y su estado entre operaciones. Los experimentos de mantenimiento y consultas dinámicas se distinguirán por sus intervalos de cronometraje, sin duplicar una modificación en el mismo recorrido.

Se ejecutarán **10 ciclos por arreglo, semilla y repetición dinámica**. Cada ciclo comenzará con **una actualización puntual**, seguida de **1.000 consultas**: primero se aplicará `update` en Segment Tree o se reconstruirá Sparse Table y, solo después, se consultará el estado vigente. Esto supone **10 modificaciones y 10.000 consultas medidas por estructura en cada repetición dinámica**. Las 1.000 consultas de cada ciclo se repartirán equilibradamente entre las longitudes de rango seleccionadas, registrando por separado el tiempo y la cantidad de consultas de cada longitud. Así se reutilizará cada estado actualizado para todos sus lotes, sin multiplicar las reconstrucciones al cambiar la longitud del rango.

No se forzará que las consultas aleatorias incluyan la posición modificada, para conservar la distribución de rangos definida. Como comprobación de corrección, después del lote se verificará también la posición actualizada mediante una consulta unitaria fuera del cronometraje.

Se conservarán muestras crudas y resúmenes CSV con **media, desviación estándar muestral, mínimo, Q1, mediana, Q3 y máximo**, identificando estructura, tamaño del arreglo, longitud del rango, semilla, repetición y ciclo cuando corresponda. Para resumir el escenario dinámico se obtendrá, en cada repetición, el tiempo medio por modificación sobre sus diez ciclos y el tiempo por consulta de cada longitud sobre sus lotes. Los ciclos no se contarán como repeticiones independientes adicionales. Las figuras mostrarán media y barras de una desviación estándar, como en el boletín

anterior; no se interpretarán como intervalos de confianza. Cada semilla conservará su resumen: las repeticiones sobre un mismo arreglo no se tratarán como arreglos independientes. Las diferencias se evaluarán junto con su dispersión: las 32 repeticiones permiten caracterizar la variabilidad, pero no garantizan por sí solas significancia estadística ni una superioridad universal a partir de las cotas asintóticas.

Antes de la ejecución oficial se realizará un **piloto**, especialmente con el arreglo de mayor tamaño, para estimar la duración de la matriz completa. El presupuesto del conjunto será de **menos de una hora**, con una proyección objetivo de **45 minutos como máximo** y **15 minutos de margen**. La estimación incluirá calentamiento, calibración, restauración y destrucción de estructuras, validación, análisis de memoria, escritura y generación de gráficos, aunque estas tareas estén fuera del cronometraje de las operaciones. Si la proyección supera el objetivo, se revisará el tamaño máximo o la cantidad de ciclos dinámicos antes de ejecutar, manteniendo las 32 repeticiones y documentando el ajuste. No se dará por garantizada la duración hasta comprobarla con el piloto.

4.3.1. Construcción inicial en el escenario estático

- **Variable:** n, cantidad de elementos del arreglo.
- **Controles:** mismos valores y pares de entrada para ambas estructuras, con las tres semillas definidas. Cada construcción se realizará desde la entrada preparada, sin incluir su generación.
- **Resultados:** tiempo por construcción frente a n, mediante tablas y gráficos. Las instancias construidas se reutilizarán para las consultas; su costo inicial no se cargará a cada consulta.

Hipótesis. Se espera que Sparse Table tenga mayor tiempo de construcción al crecer n, porque prepara una tabla de tamaño $\Theta(n \log n)$, mientras que Segment Tree construye sus nodos en $\Theta(n)$. El preprocesamiento adicional de Sparse Table puede compensarse cuando se realizan suficientes consultas estáticas, pero este beneficio se estudiará separando ambos costos, no suponiendo que ocurre para cualquier cantidad de consultas.

4.3.2. Consultas en el escenario estático

- **Variables:** tamaño del arreglo n y longitud del rango k. Se variará una manteniendo fija la otra en cada comparación.
- **Controles:** instancias ya construidas, sin modificaciones del arreglo, y mismos lotes de intervalos aleatorios. Se comparará `query_idempotent` de Sparse Table con `query` de Segment Tree.
- **Resultados:** tiempo por consulta frente a n para longitudes fijas y frente a k para tamaños de arreglo fijos. La construcción inicial quedará excluida de estos tiempos.

Hipótesis. Se espera que Sparse Table tenga menor tiempo por consulta al resolver RMQ con dos accesos y una combinación, de costo $\Theta(1)$, aprovechando su preprocesamiento. Segment Tree recorrerá los niveles necesarios para descomponer el rango, con peor caso $\Theta(\log n)$. No se presupone una diferencia apreciable para todos los rangos: los unitarios son constantes en ambas estructuras, y con k fijo la cota del recorrido del árbol no obliga a crecer con n.

4.3.3. Mantenimiento en el escenario dinámico

- **Variable:** tamaño del arreglo n, bajo modificaciones puntuales de sus valores.
- **Controles:** mismo estado inicial y misma secuencia de posiciones y nuevos valores. Se realizarán diez ciclos, cada uno con una modificación seguida de 1.000 consultas compartidas entre las longitudes evaluadas. Cada cambio deberá quedar reflejado antes de responderlas; no se agruparán modificaciones para reconstruir Sparse Table una sola vez.
- **Resultados:** tiempo por modificación frente a n. Se medirá `update` en Segment Tree y la reconstrucción en Sparse Table, excluyendo las consultas y la construcción inicial.

Hipótesis. Se espera que Segment Tree tenga menor tiempo de mantenimiento, ya que actualiza una hoja y sus ancestros en $\Theta(\log n)$, frente a reconstruir Sparse Table en $\Theta(n \log n)$. La diferencia debería aumentar al crecer el arreglo. Esta comparación expresa el costo de mantener respuestas correctas tras un cambio, no dos operaciones nativas de actualización equivalentes.

4.3.4. Consultas en el escenario dinámico

- **Variables:** tamaño del arreglo n y longitud del rango k, manteniendo fija una al estudiar la otra.
- **Controles:** mismos cambios previos y lotes de consultas. Antes de consultar, Segment Tree deberá estar actualizado y Sparse Table reconstruida, de modo que ambas representen el mismo arreglo vigente. Se usarán los lotes de 1.000 consultas de los diez ciclos del experimento de mantenimiento, sin repetir los cambios ni las reconstrucciones para obtener estas mediciones.
- **Resultados:** tiempo por consulta después de las modificaciones, separado del tiempo de mantenimiento registrado en el experimento anterior. Las respuestas deberán coincidir con el mínimo y el desempate definidos.

Hipótesis. Una vez mantenidas las estructuras, Sparse Table debería conservar su ventaja en consultas por su costo $\Theta(1)$; Segment Tree mantiene su cota de peor caso $\Theta(\log n)$. Por tanto, la ventaja esperada de Segment Tree en un entorno con cambios se atribuye al menor costo de mantenimiento, no necesariamente a una consulta aislada más rápida. Los tiempos pueden diferir de los estáticos por el estado de la caché dejado por las actualizaciones o reconstrucciones, sin que cambien estas cotas.

4.3.5. Análisis del espacio utilizado

Se analizarán las asignaciones de los constructores para los tamaños de arreglo evaluados, considerando el mismo tipo de par en ambas estructuras. Se distinguirá el almacenamiento retenido de las reservas temporales de construcción o reconstrucción, excluyendo el arreglo externo compartido. Cualquier seguimiento de asignaciones se realizará en una pasada separada, sin activarlo durante el cronometraje; estos bytes no representan la memoria residente total del proceso.

Hipótesis. Sparse Table requerirá más espacio al crecer n, por sus $n(\lfloor \log_2 n \rfloor + 1)$ elementos almacenados, frente a los $2n$ de Segment Tree. Las actualizaciones del árbol reutilizan el vector existente, mientras que la reconstrucción de la tabla requiere nuevas reservas y puede aumentar el pico si coexisten las versiones anterior y nueva. Este análisis acompañará los cuatro experimentos temporales, sin constituir un quinto experimento de tiempo.

Parámetros por concretar antes de la ejecución: tamaños de arreglos, longitudes de rangos, cantidad de consultas estáticas y criterios de calibración de sus lotes. Los diez ciclos dinámicos propuestos quedarán sujetos a la validación del presupuesto mediante el piloto, sin reducir las 32 repeticiones.

Referencias

[1] LELE. *Generic Sparse Table* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/sparse_table.cpp.

[2] LELE. *Generic Segment Tree for RMQs* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/segment_tree.cpp.

[3] Colaboradores de CP-Algorithms. *Sparse Table*. https://cp-algorithms.com/data_structures/sparse-table.html.

[4] Colaboradores de CP-Algorithms. *Segment Tree*. https://cp-algorithms.com/data_structures/segment_tree.html.

[5] AtCoder Library. *Segtree*: documentación de árboles de segmentos sobre monoides. https://atcoder.github.io/ac-library/production/document_en/segtree.html.

[6] Cppreference. *std::bit_width*. https://en.cppreference.com/w/cpp/numeric/bit_width.html.

[7] Colaboradores de CP-Algorithms. *Euclidean algorithm for computing the greatest common divisor*. https://cp-algorithms.com/algebra/euclid-algorithm.html.

[8] Cppreference. *std::lcm*. https://en.cppreference.com/w/cpp/numeric/lcm.html.

[9] Lovera, L. (LELE). (s. f.). *uhr: generic time performance tester* [Código fuente]. Plantilla de medición de la ayudantía, disponible en `utils/uhr.cpp` del repositorio de estos boletines. Referencia del autor utilizada en los boletines anteriores: https://github.com/leonardlover.